



创新创业实践

Project2

说明文档

郎泓森

202200460010

文件说明

blind_watermark 是水印所需要的库

test_image 是测试图片

watermark 是具体的代码实现

一、核心算法数学原理（基于频域水印）

1. 水印嵌入过程

令 I 为原始图像矩阵，通过变换 T （DCT 或 DWT）得到频域系数：

$$F = T(I)$$

水印文本 W 被转换为二进制序列 $b = \{b_1, b_2, \dots, b_n\}$ ，通过伪随机序列 p （由 password_wm 生成）选择嵌入位置：

$$F'_k = F_k + \alpha \cdot b_i \cdot p_j$$

其中：

α ：嵌入强度因子（控制不可见性与鲁棒性的平衡）

p_j ：均值为 0 的伪随机噪声序列（ $E[p_j] = 0$ ）

逆变换生成含水印图像：

$$I' = T^{-1}(F')$$

2. 水印提取过程

对含水印图像 I' 进行相同变换：

$$\hat{F} = T(I')$$

使用相同伪随机序列 p 计算相关性：

$$c_i = \frac{1}{m} \sum_{j=1}^m \hat{F}_{k_j} \cdot p_j$$

其中 k_j 是预设的嵌入位置索引。提取的比特位由相关性符号判定：

$$\hat{b}_i = \begin{cases} 1 & \text{if } c_i > \tau \\ 0 & \text{otherwise} \end{cases}$$

2

阈值 τ 通常设为 0。最终将二进制序列解码为文本。

关键数学特性

1. 不可见性

通过限制修改幅度 α 保证：

$$\|I' - I\|_{\infty} < \epsilon (\epsilon \text{ 为视觉不可感知阈值})$$

2. 鲁棒性原理

频域系数在攻击下的稳定性：

几何攻击（翻转/平移）

导致相位偏移，但频域幅度变化有限

信号处理（对比度/亮度调整）

可建模为全局线性变换： $I'' = a \cdot I' + b$

提取时相关性保持： $c^i \approx a \cdot c_i$

裁剪攻击

保留频域低频分量（能量集中区域），水印仍可部分恢复

二、代码解释及运行结果

完整代码解释

1. 整体结构

代码实现了一个完整的数字水印测试框架，包括：

水印嵌入功能、水印提取功能、8 种图像攻击的鲁棒性测试以及测试结果分析

通过导入 `blind_watermark` 水印库进行编程。

2. 模块导入

```
from blind_watermark import WaterMark
from PIL import Image, ImageOps, ImageEnhance
import numpy as np
import os
```

`blind_watermark` 是核心水印库，包括实现盲水印的核心算法

`PIL`（`Pillow`）库用来图像处理（翻转、裁剪、色彩调整等）

`numpy` 库高效数组运算（用于图像平移）

os 库用来创建目录和文件操作，生成攻击后的图片

3. 水印嵌入函数

```
def embed_watermark(input_path='test_image.jpg', output_path='embedded.png', wm_text='SECRET'):
    bwm = WaterMark(password_img=1, password_wm=1)
    bwm.read_img(input_path)
    bwm.read_wm(wm_text, mode='str')
    bwm.embed(output_path)
    len_wm = len(bwm.wm_bit)
    print(f'水印长度: {len_wm}')
    return len_wm
```

embed_watermark 函数实现了水印嵌入，使用双密码系统（password_img 和 password_wm）增强安全性，将水印文本自动转换为二进制序列，并返回水印长度用于后续提取。

4. 水印提取函数

```
def extract_watermark(img_path, wm_length, mode='str', attack_type=None, attack_params=None):
    temp_path = "temp_recovered.png"

    # 根据攻击类型进行逆变换[1,2](@ref)
    if attack_type == 'flip_horizontal':
        img = Image.open(img_path)
        img_recovered = img.transpose(Image.FLIP_LEFT_RIGHT)
        img_recovered.save(temp_path)
        img_path = temp_path
        print("已执行水平翻转逆变换")

    elif attack_type == 'flip_vertical':
        img = Image.open(img_path)
        img_recovered = img.transpose(Image.FLIP_TOP_BOTTOM)
        img_recovered.save(temp_path)
        img_path = temp_path
        print("已执行垂直翻转逆变换")
```

extract_watermark 函数用来从图片中提取水印，必须使用相同的密码才能正确提取，需要知道水印的二进制长度（wm_length），支持返回字符串格式（mode='str'），其中对于水平翻转、垂直翻转以及平移等攻击，在提取之前需要先进行逆变换。

5. 鲁棒性测试框架

```

def test_robustness(original_path, wm_length, output_dir='attacked_images'):
    os.makedirs(output_dir, exist_ok=True)

    # 记录结果
    results = []

    # ===== 攻击函数组 =====
    def add_attack(name, func, attack_type=None, attack_params=None):
        nonlocal results
        attacked_path = f"{output_dir}/{name}.png"
        func(original_path, attacked_path)

        try:
            # 提取时传递攻击类型和参数[4](@ref)
            extracted = extract_watermark(
                attacked_path,
                wm_length,
                attack_type=attack_type,
                attack_params=attack_params
            )
            results.append((name, extracted))
            print(f"{name}: 提取结果 -> {extracted}")
        except Exception as e:
            results.append((name, f"提取失败: {str(e)}"))
            print(f"{name}: 提取失败 - {str(e)}")

```

test_robustness 用来测试该水印库嵌入水印对各种攻击的鲁棒性。

6. 图像攻击类型详解

(1) 水平翻转

水平翻转（添加逆变换支持）

```

def flip_horizontal(in_path, out_path):
    img = Image.open(in_path)
    img_flipped = img.transpose(Image.FLIP_LEFT_RIGHT)
    img_flipped.save(out_path)

```

```
add_attack("水平翻转", flip_horizontal, attack_type='flip_horizontal')
```

测试水印对镜像变换的抵抗能力

(2) 垂直翻转

垂直翻转（添加逆变换支持）

```

def flip_vertical(in_path, out_path):
    img = Image.open(in_path)
    img_flipped = img.transpose(Image.FLIP_TOP_BOTTOM)
    img_flipped.save(out_path)

```

```
add_attack("垂直翻转", flip_vertical, attack_type='flip_vertical')
```

测试水印对垂直翻转变换的抵抗能力

(3) 平移攻击

```
# 平移（添加逆变换支持）
def shift_image(in_path, out_path, dx=50, dy=30):
    img = Image.open(in_path)
    img_np = np.array(img)
    shifted = np.roll(img_np, (dy, dx), axis=(0, 1))
    Image.fromarray(shifted).save(out_path)

add_attack("平移攻击", shift_image, attack_type='shift', attack_params={'dx': 50, 'dy': 30})
```

测试空间位移对水印的影响

(4) 裁剪攻击

```
# 裁剪（不需要逆变换）
def crop_image(in_path, out_path, ratio=0.8):
    img = Image.open(in_path)
    w, h = img.size
    new_w, new_h = int(w * ratio), int(h * ratio)
    img_cropped = img.crop((0, 0, new_w, new_h))
    img_padded = Image.new(img.mode, (w, h), color=(0, 0, 0))
    img_padded.paste(img_cropped, (0, 0))
    img_padded.save(out_path)

add_attack("裁剪攻击", crop_image)
```

模拟图像部分内容丢失的场景

(5) 对比度增强

```
# 对比度增强（不需要逆变换）
def enhance_contrast(in_path, out_path, factor=2.0):
    img = Image.open(in_path)
    enhancer = ImageEnhance.Contrast(img)
    img_contrast = enhancer.enhance(factor)
    img_contrast.save(out_path)

add_attack("对比度增强", enhance_contrast)
```

(6) 对比度减弱

```
# 对比度减弱（不需要逆变换）
def reduce_contrast(in_path, out_path, factor=0.5):
    img = Image.open(in_path)
    enhancer = ImageEnhance.Contrast(img)
    img_contrast = enhancer.enhance(factor)
    img_contrast.save(out_path)

add_attack("对比度减弱", reduce_contrast)
```

(7) 颜色偏移

```
# 颜色平衡破坏（不需要逆变换）
def color_shift(in_path, out_path):
    img = Image.open(in_path)
    r, g, b = img.split()
    r = r.point(lambda x: min(x * 1.5, 255))
    img_shifted = Image.merge('RGB', (r, g, b))
    img_shifted.save(out_path)

add_attack("颜色偏移", color_shift)
```

测试颜色平衡改变对频域水印的影响

7. 主程序流程

```
# ===== 主程序 =====
if __name__ == "__main__":
    # 嵌入水印
    wm_text = 'SECRET'
    len_wm = embed_watermark(output_path='embedded.png', wm_text=wm_text)

    # 直接提取测试
    print("\n=== 直接提取测试 ===")
    extracted = extract_watermark('embedded.png', len_wm)
    print(f"原始水印: {wm_text}")
    print(f"提取结果: {extracted}")

    # 进行鲁棒性测试
    print("\n=== 鲁棒性测试开始 ===")
    test_results = test_robustness('embedded.png', len_wm)

    # 打印最终结果
    print("\n=== 最终测试结果 ===")
    for attack, result in test_results:
        print(f"{attack:<12} -> {result}")
```

首先在嵌入阶段，读取原始图像 `test_image.jpg`，将文本"SECRET"转换为二进制水印，使用频域变换（DCT/DWT）嵌入水印，保存含水印图像 `embedded.png`，然后记录水印长度（如 48 位）

基本测试阶段，从 `embedded.png` 直接提取水印，然后验证提取结果与原始文本是否一致。接下来进行鲁棒性测试（核心），对含水印图像进行 8 种攻击，每种攻击保存修改后的图像，接下来代码尝试从被攻击图像中提取水印，并记录每种攻击的提取结果。

最后在结果分析阶段，水平打印所有测试结果，格式：攻击类型 -> 提取结

果/失败原因。

运行结果

```
=== 鲁棒性测试开始 ===  
已执行水平翻转逆变换  
水平翻转：提取结果 -> SECRET  
已执行垂直翻转逆变换  
垂直翻转：提取结果 -> SECRET  
已执行平移逆变换(dx=-50, dy=-30)  
平移攻击：提取结果 -> SECRET  
裁剪攻击：提取结果 -> SECRET  
对比度增强：提取结果 -> SECRET  
对比度减弱：提取结果 -> SECRET  
颜色偏移：提取结果 -> SECRET
```

```
=== 最终测试结果 ===  
水平翻转          -> SECRET  
垂直翻转          -> SECRET  
平移攻击          -> SECRET  
裁剪攻击          -> SECRET  
对比度增强        -> SECRET  
对比度减弱        -> SECRET  
颜色偏移          -> SECRET
```

可以看到，该代码具有较高的鲁棒性；该图片水印能够抵御一定的水平、垂直翻转、平移以及裁剪、对比度增强、对比度减弱、颜色偏移等攻击。